NATIONAL FORENSIC SCIENCE UNIVERSITY

PAPER-II: VULNERABILITY ASSESSMENT AND PENETRATION TESTING

OWASP TOP 10:2025

SUBMITTED BY: PRIYA NAUTIYAL

MSc FORENSIC SCIENCE (CYBER SPECIALISATION)

250222001001

OWASP Top 10: 2025

The **OWASP Top 10: 2025** is a list published by the Open Worldwide Application Security Project (OWASP) that identifies the most critical security risks for web applications. It is used by developers, cybersecurity professionals, penetration testers, and organizations to understand common web application vulnerabilities and improve application security.

A01: Broken Access Control occurs when an application does not properly restrict what users are allowed to access or perform. An attacker may be able to access another user's information, modify restricted data, or perform administrative actions without proper authorization.

A02: Security Misconfiguration happens when security settings are incorrectly configured or left at unsafe defaults. Examples include unnecessary services being enabled, default passwords being used, excessive permissions being granted, or detailed error messages being exposed to users.

A03: Software Supply Chain Failures refers to security problems involving third-party libraries, dependencies, development tools, build systems, and other components used to create or deploy an application. A compromised or vulnerable dependency can introduce security risks into an otherwise secure application.

A04: Cryptographic Failures occur when sensitive information is not properly protected using cryptography. Examples include weak encryption, poor key management, or transmitting sensitive information without adequate protection.

A05: Injection occurs when an application accepts untrusted input and incorrectly interprets it as commands or queries. Common examples include SQL injection, command injection, and other forms of injection that can allow attackers to manipulate application behavior.

A06: Insecure Design refers to security weaknesses that exist because security was not properly considered during the design of an application. Unlike a simple configuration problem, insecure design often requires changing the application's architecture, logic, or security controls.

A07: Authentication Failures occur when an application does not properly verify the identity of users. Weak passwords, poor session management, inadequate authentication controls, and improperly implemented login mechanisms can allow attackers to gain unauthorized access.

A08: Software or Data Integrity Failures occur when applications trust software, updates, plugins, dependencies, or data without properly verifying their integrity. Attackers may exploit this weakness by modifying or replacing trusted software or data.

A09: Security Logging and Alerting Failures happen when important security events are not properly recorded, monitored, or reported. Without effective logging and alerting, organizations may fail to detect attacks or may discover them only after significant damage has occurred.

A10: Mishandling of Exceptional Conditions occurs when an application handles errors, unexpected input, or unusual situations insecurely. Poor error handling can cause applications to expose sensitive information, bypass security controls, crash unexpectedly, or enter unsafe states.

1.Broken Access Control

Broken Access Control happens when an application fails to properly enforce what a user is allowed to access or do.

A common example is an **IDOR (Insecure Direct Object Reference)**

If the logged-in user is allowed to view their own profile but changing the ID reveals another user's profile, the application has an access-control vulnerability.

The important point is that **changing the ID itself isn't vulnerability**. The vulnerability is that the server fails to verify whether the current user is authorized to access object `101`.

Open Burp Suite

Start **Burp Suite Community Edition** or another authorized Burp installation.

Go to:

Proxy → Proxy settings

Make sure Burp's proxy listener is running, normally on:

```
127.0.0.1:8080
```

Configure your browser to send its traffic through Burp.

Then open your authorized lab application.

Log in normally

First, log in as the normal test user provided by the lab.

For example, imagine the application gives you:

```
Username: testuser
Password: ********
```

After logging in, browse the application normally.

Look for functionality such as:

- Profile
- Account
- Orders
- Messages
- Files
- User IDs
- Admin functions
- Edit/delete functionality

These are common places where access-control problems can occur.

Capture a request with Burp

Go to:

Proxy → HTTP history

You'll see requests generated by your browser.

For example, you might find:

```
GET /profile?id=123 HTTP/1.1
Host: lab. example
Cookie: session=...
```

The interesting part is:

```
id=123
```

This could be an object identifier.

Send the request to Repeater

Right-click the request and select:

Send to Repeater

Then go to:

Repeater

Burp will allow you to modify and resend the request.

This is extremely useful for testing access-control behavior because you can change one parameter at a time and compare the responses.

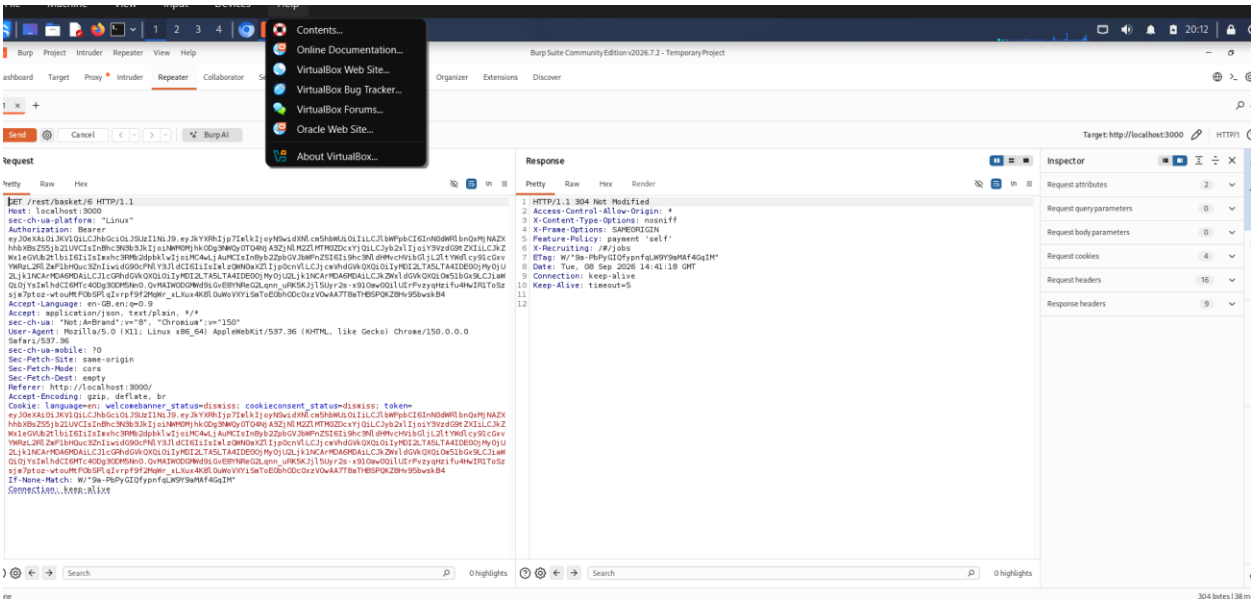


Fig. 1: picture showing normal response

Change the object identifier

Now, in the **authorized lab**, change:

to another test object's identifier:

Send the request again.

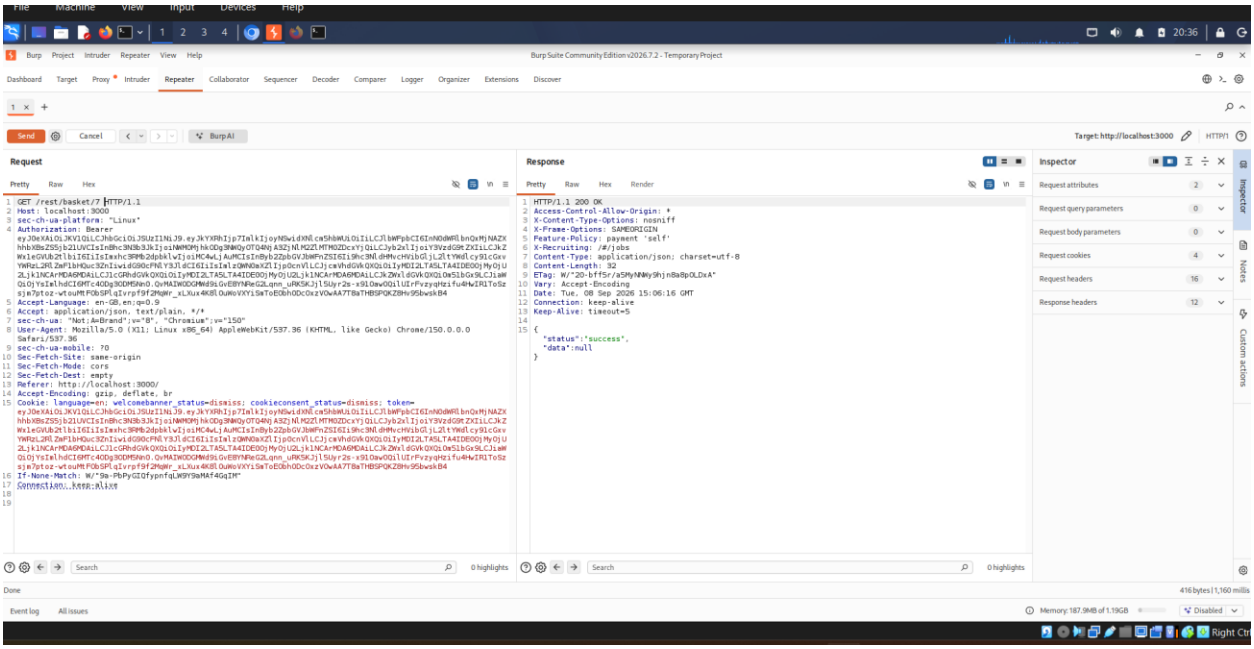


Fig. 2:picture showing modified request

OBSERVATION: During the testing of Broken Access Control using OWASP methodology and Burp Suite, the application's access-control mechanism was examined by intercepting HTTP requests and analyzing the parameters used to identify resources. The captured requests were sent to Burp Suite Repeater, where the resource identifier was modified in the authorized testing environment. It was observed that changing the identifier allowed access to a resource belonging to another test user without proper authorization. The server returned a successful response instead of denying the request. This indicates a potential Broken Access Control vulnerability, specifically an Insecure Direct Object Reference (IDOR), because the application did not adequately verify whether the authenticated user was authorized to access the requested resource.

2.SECURITY MISCONFIGURATION

Procedure: Security Misconfiguration using OWASP and Burp Suite

1. **Start Burp Suite** and configure the browser to use Burp as a proxy.
2. **Open the authorized web application** and browse through its pages so Burp can capture the traffic.

3. Go to **Proxy → HTTP History** in Burp Suite and review the captured requests and responses.
4. **Check the HTTP response headers** for missing or incorrectly configured security headers such as CSP, HSTS, and X-Content-Type-Options.
5. **Check for information disclosure**, such as server versions, framework details, or unnecessary technical information in responses.
6. Send interesting requests to **Burp Repeater** and observe how the application handles invalid or unexpected requests.
7. **Check error messages** to determine whether they reveal stack traces, file paths, database information, or other internal details.
8. Check whether **directory listing, backup files, test pages, or unnecessary administrative interfaces** are exposed.
9. Review the configuration of **HTTP methods and HTTPS/security settings** where applicable.
10. **Record the findings** with screenshots/evidence and compare them with **OWASP A02:2025 – Security Misconfiguration**.
11. Finally, provide **remediation recommendations** and retest the application after the configuration has been corrected.

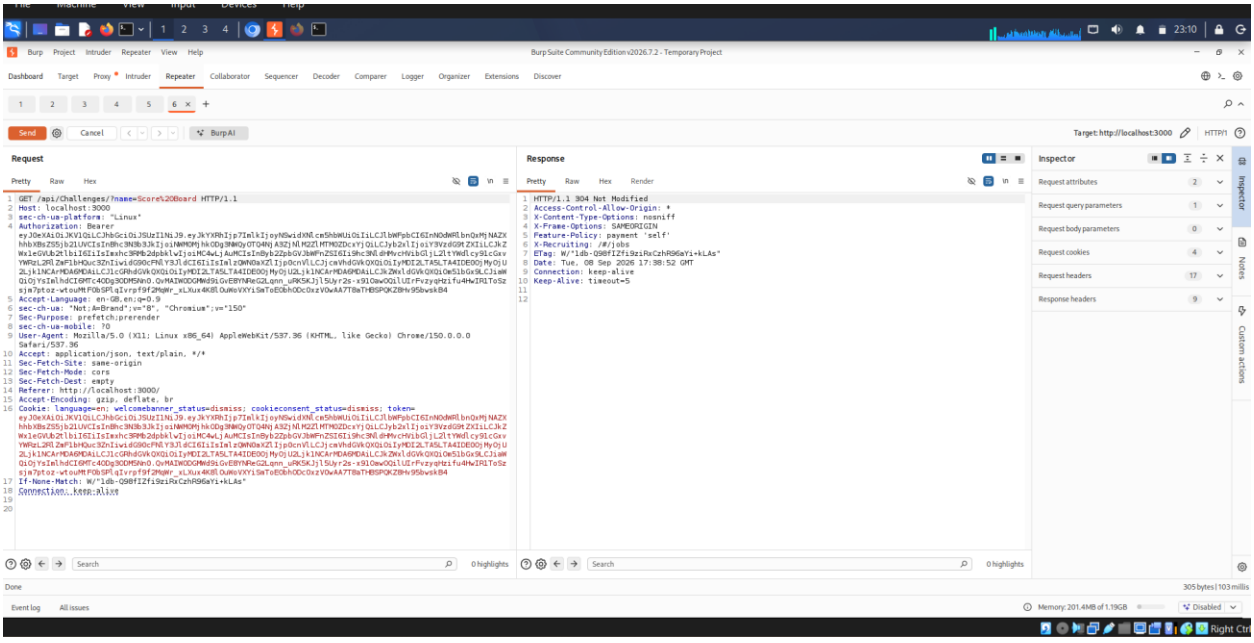


Fig. 3:picture showing normal response

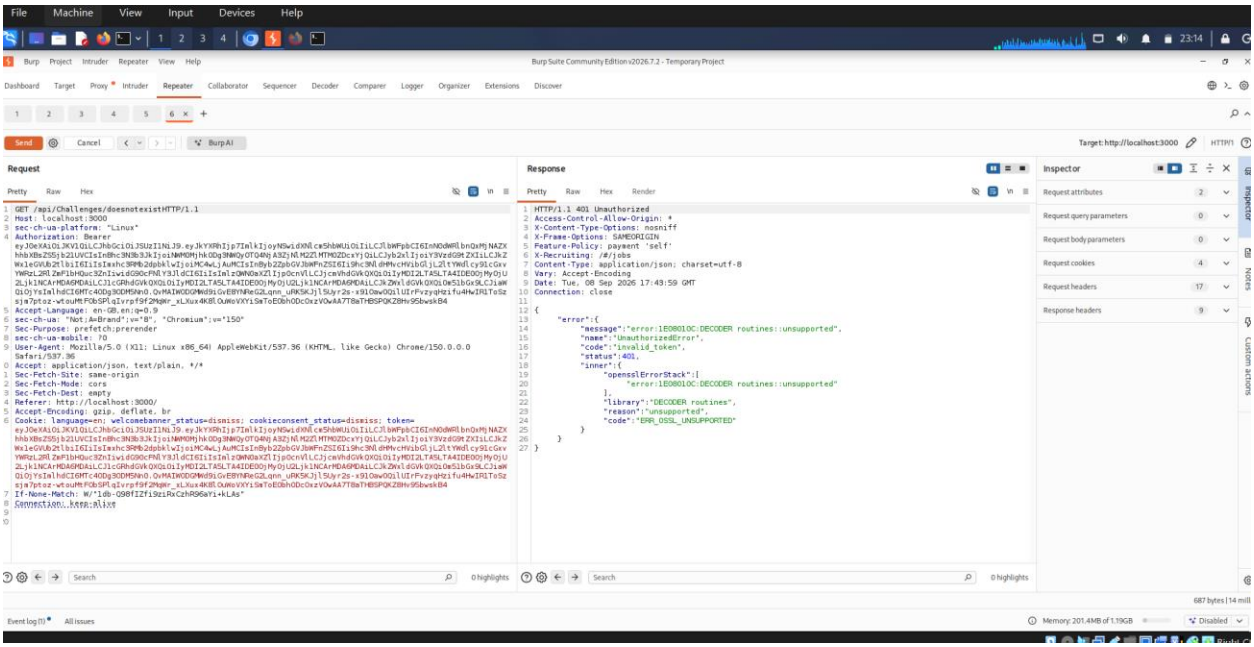
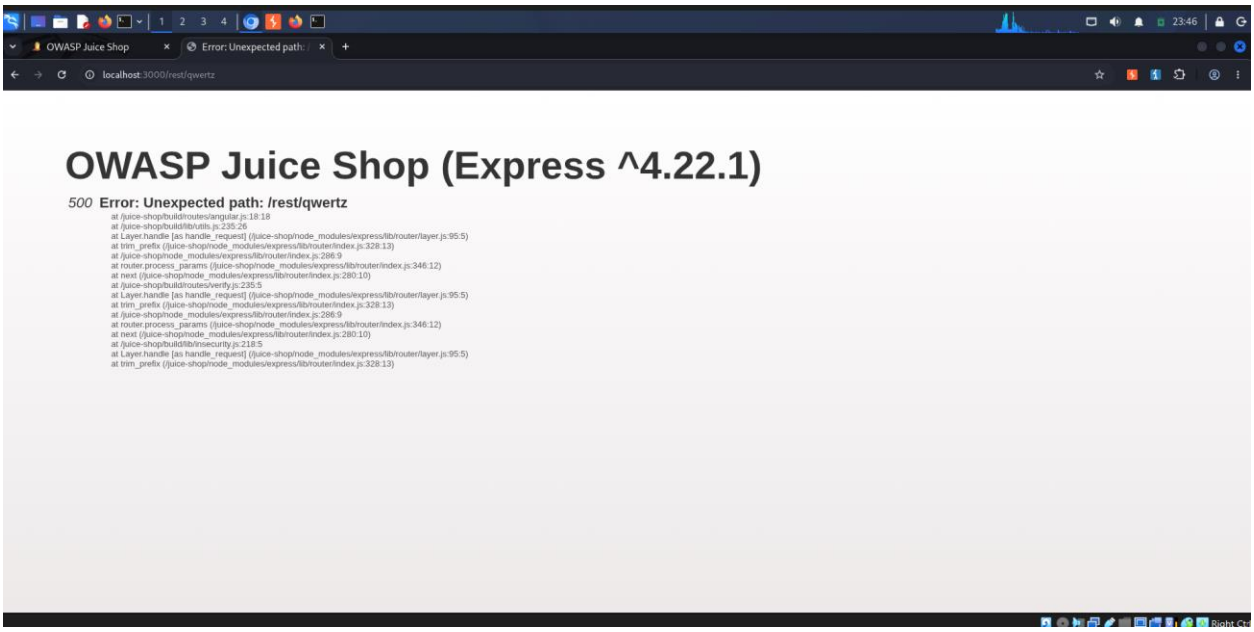


Fig. 4: picture showing modified response



Observation

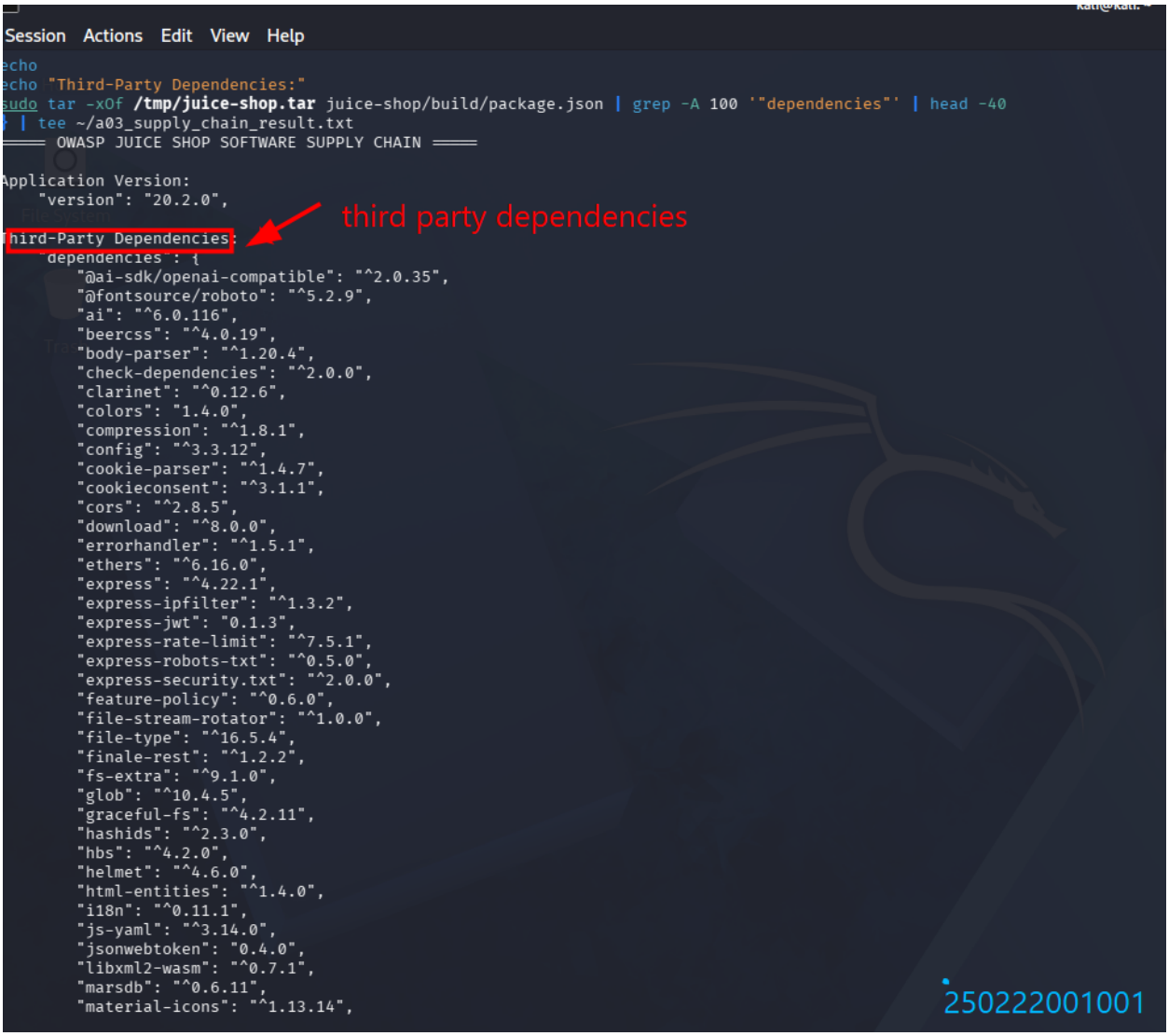
The application was tested using Burp Suite by intercepting and analyzing HTTP requests and responses. Security headers, error handling, information disclosure, exposed resources, and other configuration settings were reviewed. Any insecure or unnecessary configuration

identified was classified under **OWASP A02:2025 – Security Misconfiguration** and documented with appropriate remediation.

3.SOFTWARE SUPPLY CHAIN FAILURE

Procedure: Software Supply Chain Failure

1. **Identify the software and its dependencies** used by the application.
2. **Review third-party libraries, packages, plugins, and components** for outdated or vulnerable versions.
3. Use a security tool such as **OWASP Dependency-Check** or the application's package manager to scan dependencies.
4. **Check the source and integrity of dependencies**, ensuring they come from trusted and authorized repositories.
5. Review whether dependencies have **known vulnerabilities, outdated versions, or suspicious/unapproved components**.
6. Verify that dependency updates are performed through a **controlled and secure process**.
7. Check whether the application uses **lock files, dependency version controls, and integrity checks** to prevent unexpected package changes.
8. **Document any vulnerable or unauthorized dependency** identified during the assessment.
9. Compare the findings with **OWASP Software Supply Chain Failures** and determine the associated risk.
10. **Update, remove, or replace affected dependencies** and perform a rescan to verify that the issue has been resolved.



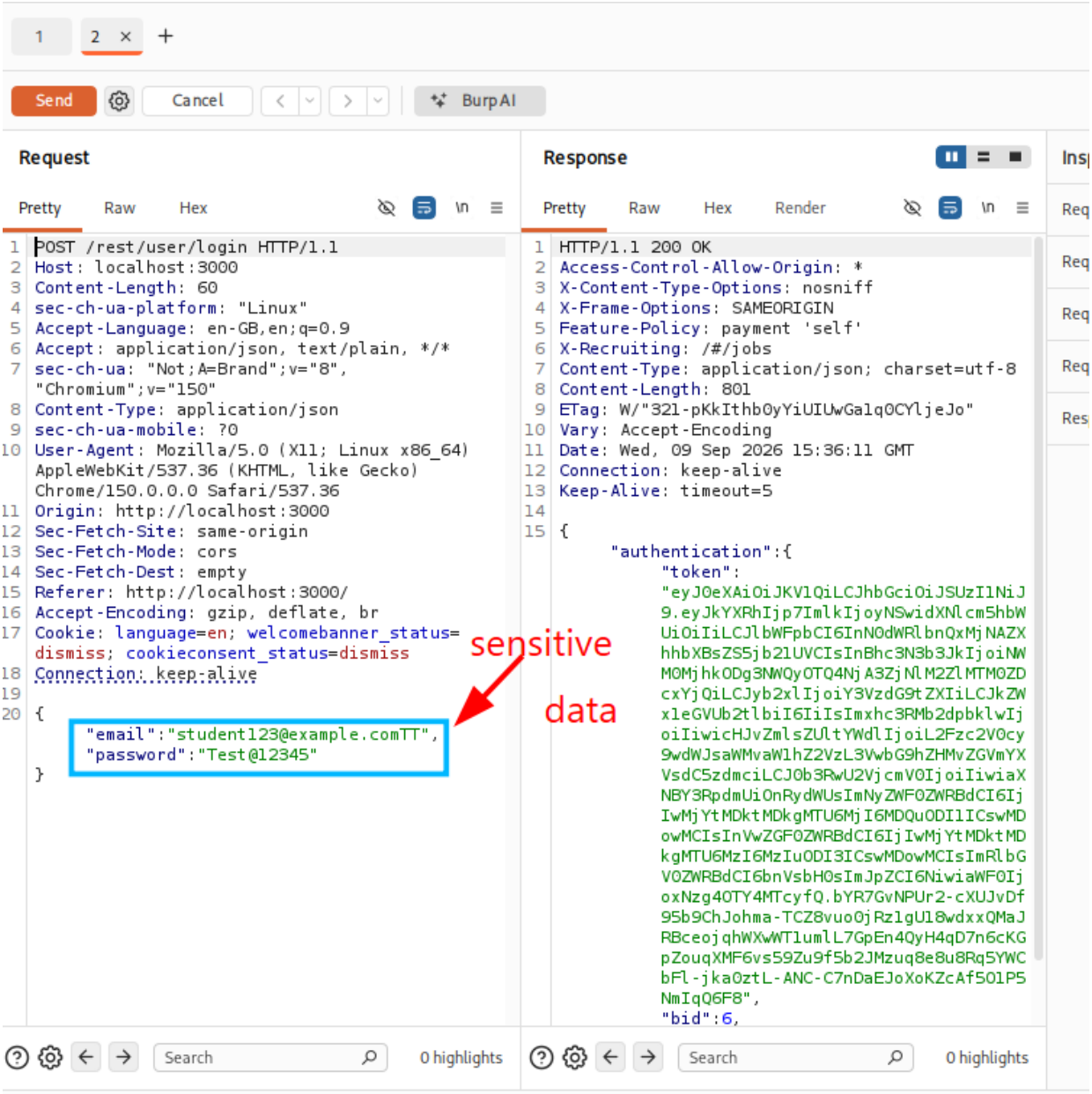
Observation – Software Supply Chain Failure

During the practical, it was observed that software applications can become vulnerable through **third-party libraries, dependencies, packages, or external components** used in the application. By examining the OWASP Juice Shop and its dependencies, it was understood that outdated or compromised components may introduce security vulnerabilities into the application. The practical demonstrated that organizations must regularly **identify dependencies, check their versions, scan them for known vulnerabilities, and update them from trusted sources**. Proper dependency management, software composition analysis, integrity verification, and timely patching are important measures for preventing software supply chain failures.

4.CRYPTOGRAPHIC FAILURES

Procedure:

1. Start OWASP Juice Shop and open it in the browser.
2. Open Burp Suite and configure the browser proxy.
3. Turn **Intercept ON** in Burp Suite.
4. Open the Juice Shop login page and enter test credentials.
5. Submit the login request and capture it in Burp Suite.
6. Inspect the request and response for sensitive information such as credentials, tokens, and cookies.
7. Check whether sensitive data is properly protected and whether cookies contain security attributes such as `Secure` and `HTTP Only`.



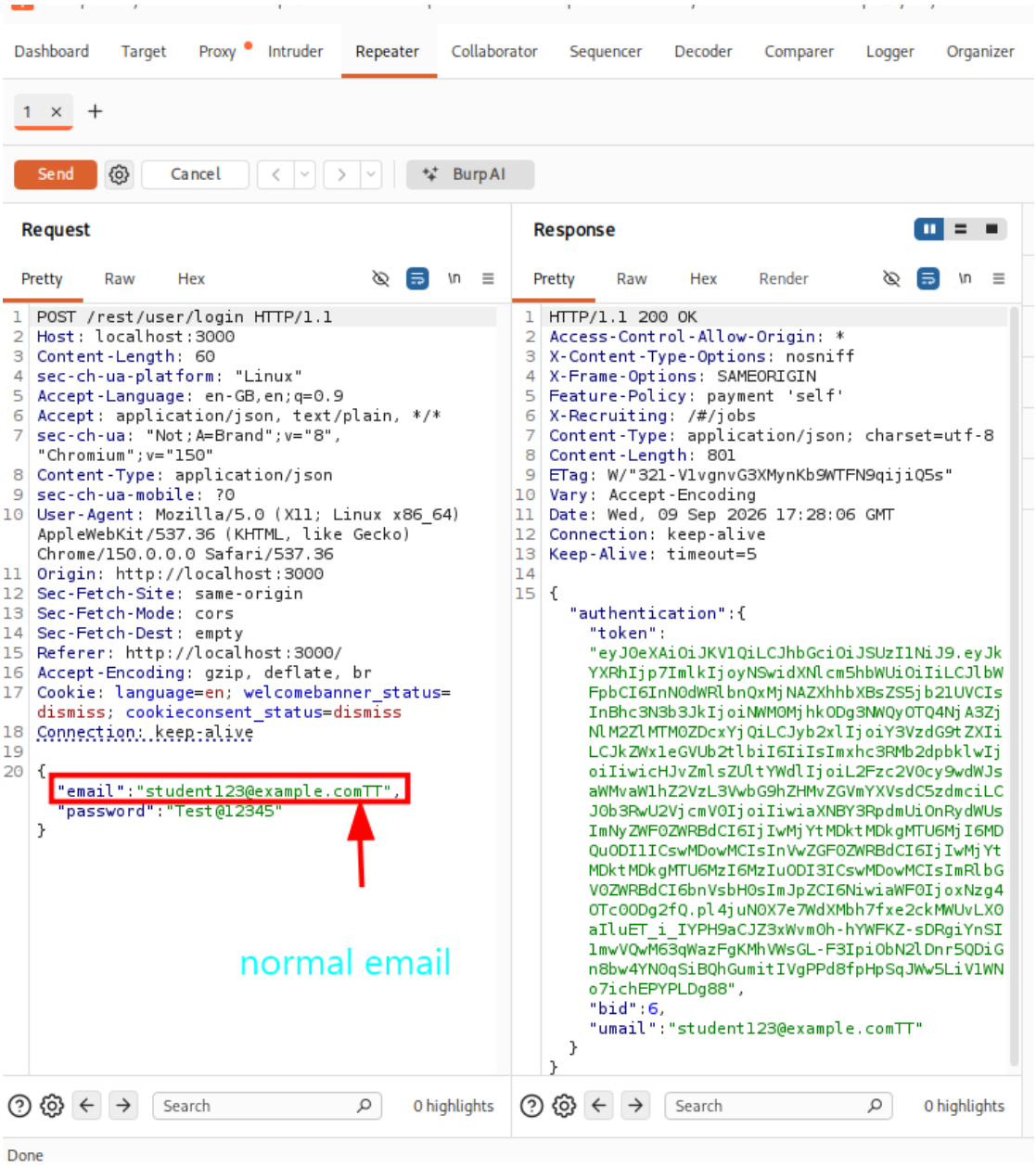
Observation:

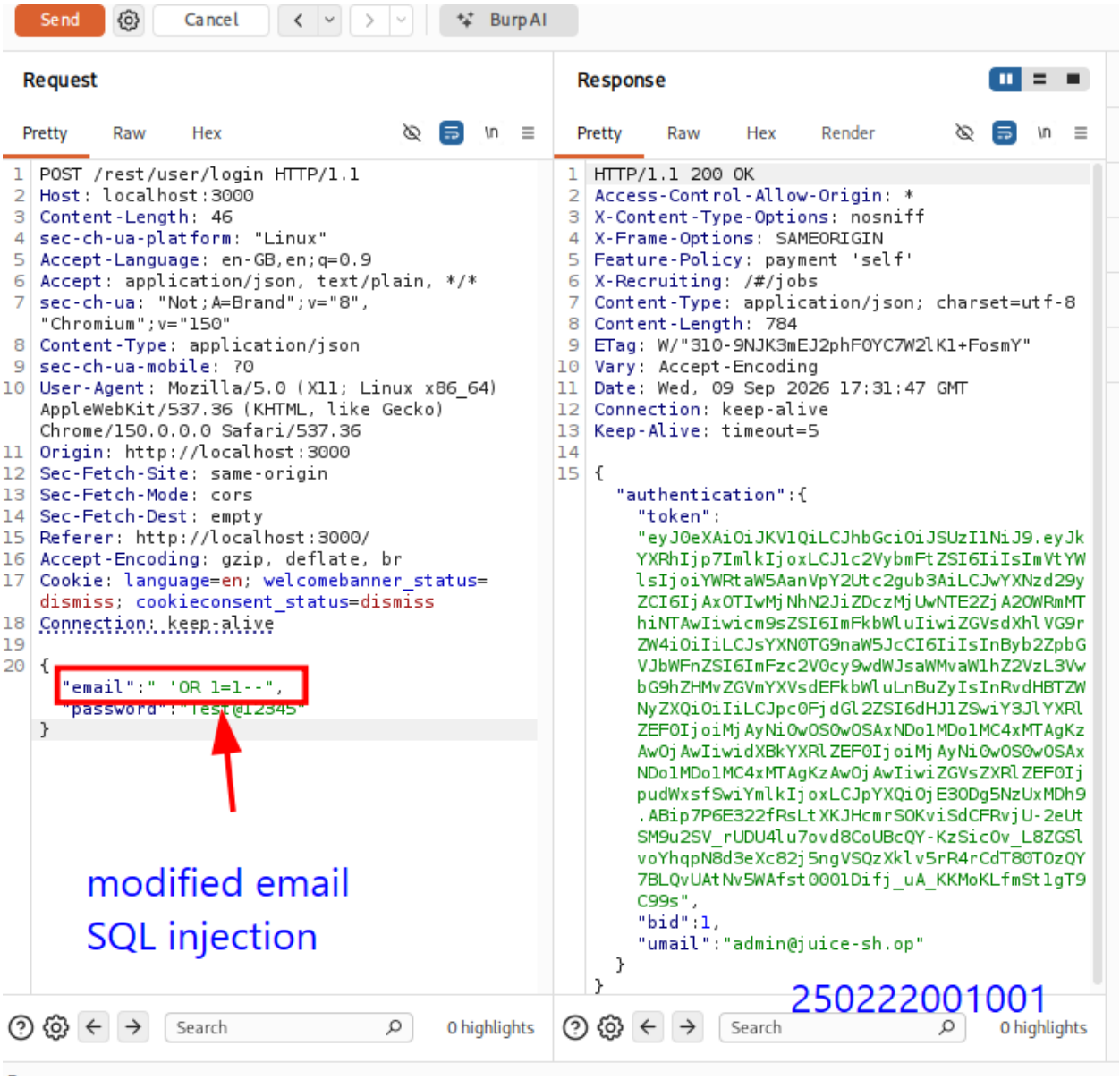
Burp Suite allowed the HTTP requests and responses of Juice Shop to be inspected. Sensitive information and security-related parameters could be observed, demonstrating the importance of proper encryption and secure handling of sensitive data.

5.INJECTION

Procedure

1. Start **OWASP Juice Shop** and open it in the browser.
2. Open **Burp Suite** and configure the browser to use Burp Proxy.
3. Turn **Intercept ON**.
4. Open the Juice Shop login/search functionality.
5. Enter test input and submit the request.
6. Capture the request in **Burp Suite → Proxy → HTTP history**.
7. Send the request to **Repeater** and modify the input parameter with a harmless SQL-injection test such as: `' OR '1'='1`
8. Send the modified request and compare the response with the normal request.
9. Observe whether the application behaves unexpectedly, such as accepting invalid input or returning unusual results.


Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
1
Send
Cancel
BurpAI
Request
Pretty
Raw
Hex
POST /rest/user/login HTTP/1.1
Host: localhost:3000
Content-Length: 60
sec-ch-ua-platform: "Linux"
Accept-Language: en-GB,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Origin: http://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
Connection: keep-alive
{
"email":"student123@example.comTT",
"password":"Test@12345"
}
normal email
Response
Pretty
Raw
Hex
Render
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 801
ETag: W/"321-V1vgnvG3XMynKb9WTFN9qijiQ5s"
Vary: Accept-Encoding
Date: Wed, 09 Sep 2026 17:28:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{
"authentication":{
"token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InN0dWRlbnQxMjNAZXhhbXBsZS5jb21lUVCIsInBhc3N3b3JkIjoiNWM0MjhkODg3NWQyOTQ4NjA3ZjNlM2ZlMTM0ZDcxYjQiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWxleGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiIiwicHJvZmlsZUltYWdlIjoiL2Fzc2V0cy9wdWJsaWMvaW1hZ2VzL3VwbG9hZHMvZGVmYXVsdC5zdmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjYtMDktMDkgMTU6MjI6MDQuODIlICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjYtMDktMDkgMTU6MzI6MzIuODI3ICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImJpZCI6NiwiaWF0IjoxNzg4OTc0ODg2fQ.pl4juNOX7e7WdXMbh7fxe2ckMWUvLX0aIluET_i_IYPH9aCJZ3xWvm0h-hYWFKZ-sDRgiYnSIlmwVQwM63qWazFgKMhVWsGL-F3IpiObN2lDnr5QDiGn8bw4YN0qSiBQhGumitIVgPPd8fpHpSqJWw5LiVlWNo7ichEPYPLDg88",
"bid":6,
"umail":"student123@example.comTT"
}
}
Search
0 highlights
Search
0 highlights
Done



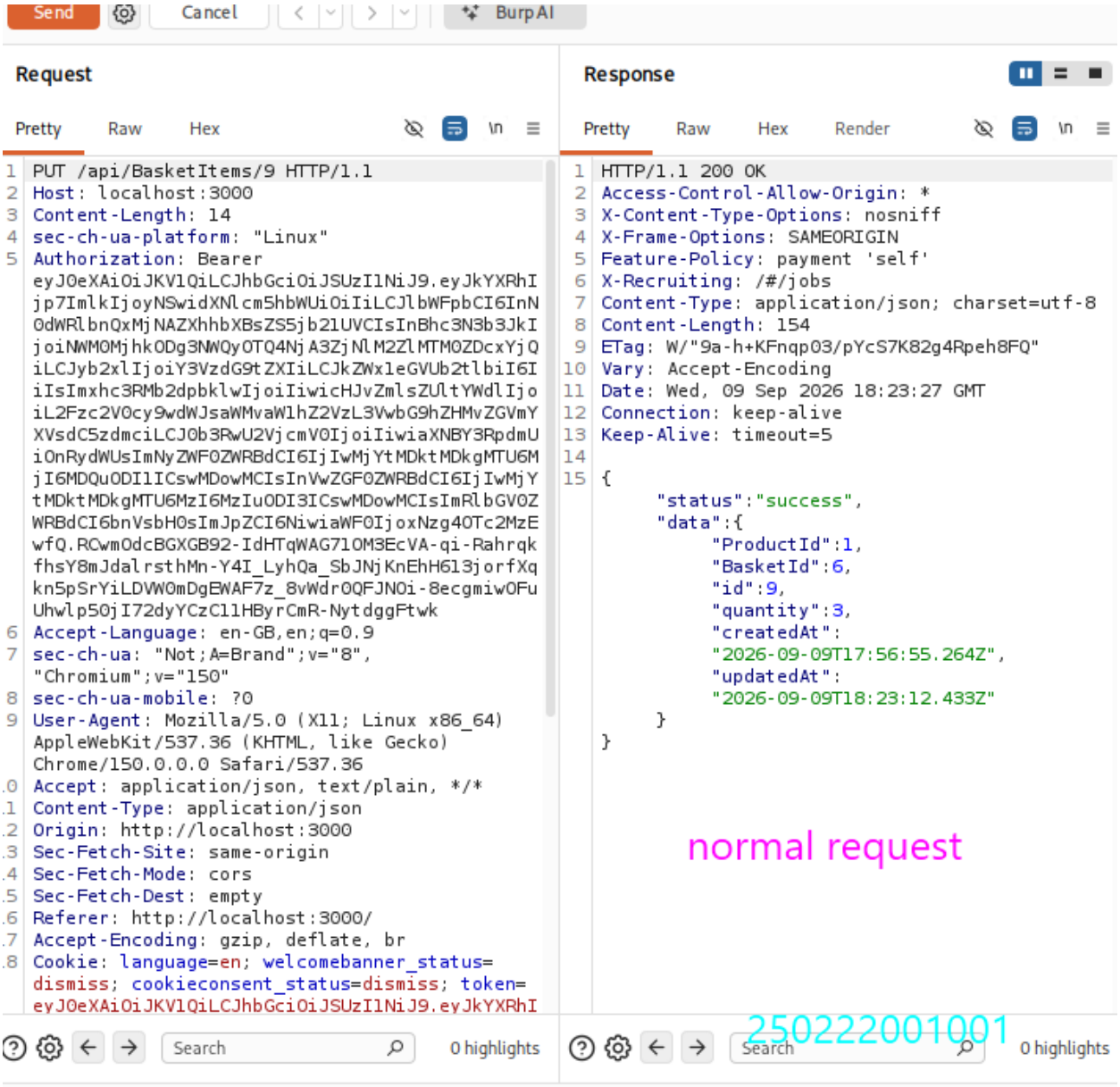
Observation

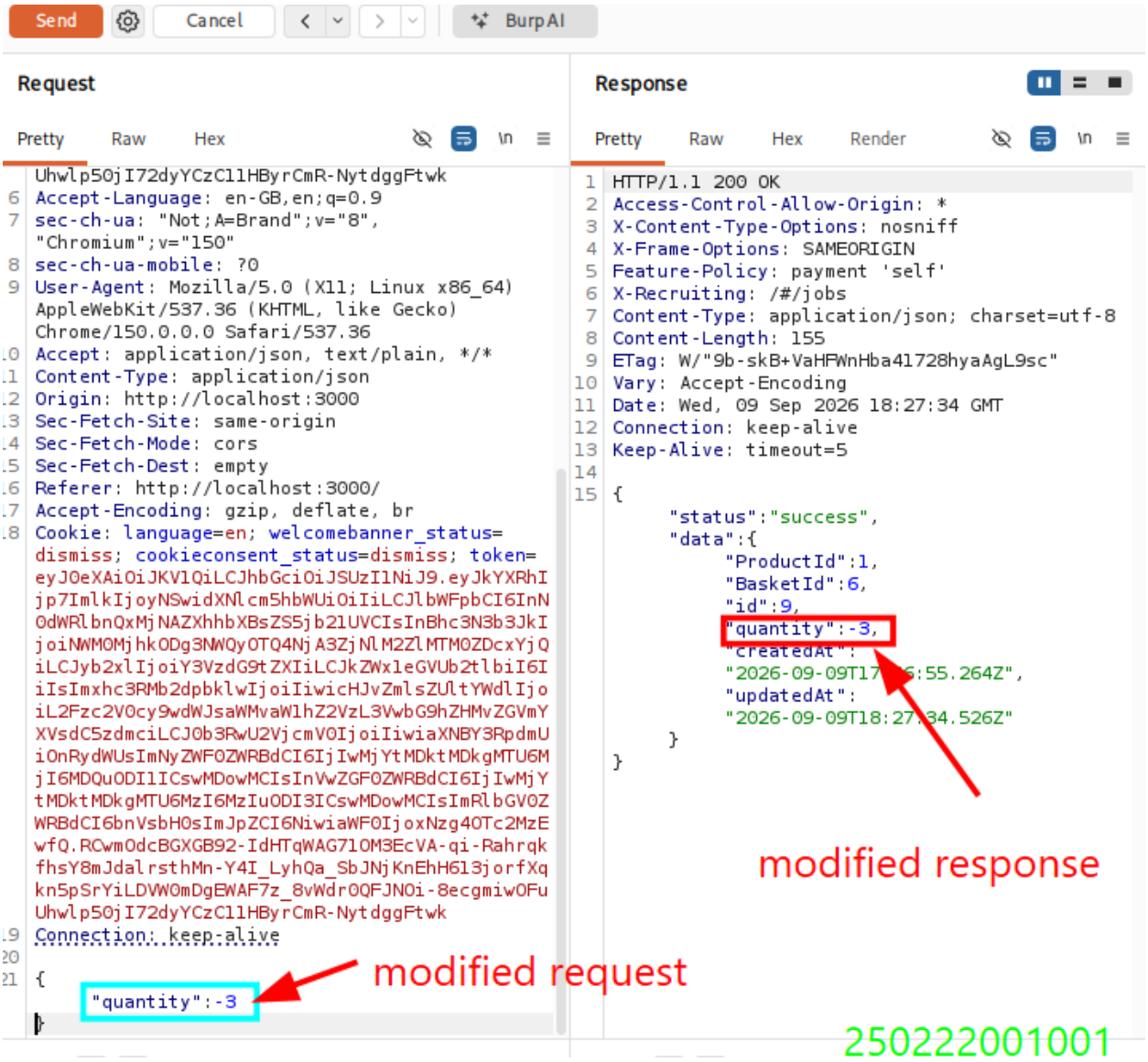
It was observed that user-supplied input is processed by the application and can be inspected and modified using Burp Suite. Improper input validation and unsafe handling of database queries can allow **SQL Injection**, potentially affecting authentication or database operations.

6.INSECURE DESIGN

Procedure

1. Start **OWASP Juice Shop** and open it in the browser.
2. Open **Burp Suite** and configure the browser proxy.
3. Turn **Intercept ON**.
4. Perform a normal application function, such as login or accessing an account-related feature.
5. Capture the request in **Proxy → HTTP history**.
6. Send the request to **Repeater**.
7. Modify parameters or request values and resend the request.
8. Compare the modified response with the normal response.
9. Observe whether the application properly enforces its intended security rules and restrictions.


Send
Cancel
Burp AI
Request
Pretty
Raw
Hex
\n
PUT /api/BasketItems/9 HTTP/1.1
Host: localhost:3000
Content-Length: 14
sec-ch-ua-platform: "Linux"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhI
jp7ImlkIjoyNSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InN
0dWRlbnQxMjNAZXhhbXBsZS5jb21UVCIsInBhc3N3b3JkI
joiNWM0MjhkODg3NWQyOTQ4NjA3ZjNlM2ZlMTM0ZDcxYjQ
iLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6I
iIsImxhc3RMb2dpbklwIjoiIiwicHJvZmlsZUltYWdlIjo
iL2Fzc2V0cy9wdWJsaWMvaW1hZ2VzL3VwbG9hZHMvZGVmY
XVsdC5zdmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmU
iOnRydWUsImNyZWF0ZWRBdCI6IjIwMjYtMDktMDkgMTU6M
jI6MDQuODIlICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjY
tMDktMDkgMTU6MzI6MzIuODI3ICswMDowMCIsImRlbGV0Z
WRBdCI6bnVsbH0sImJpZCI6NiwiaWF0IjoxNzg4OTc2MzE
wfQ.RCwmOdcBGXGB92-IdHTqWAG71OM3EcVA-qi-Rahrqk
fhsY8mJdalrsthMn-Y4I_LyhQa_SbJNjKnEhH613jorfXq
kn5pSrYiLDVW0mDgEWAF7z_8vWdr0QFJNOi-8ecgmiwOFu
Uhwlp50jI72dyYCzC11HByrCmR-NytdggFtwk
Accept-Language: en-GB,en;q=0.9
sec-ch-ua: "Not;A=Brand";v="8",
"Chromium";v="150"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/150.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
Origin: http://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=
dismiss; cookieconsent_status=dismiss; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhI
Search
0 highlights
Response
Pretty
Raw
Hex
Render
\n
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 154
ETag: W/"9a-h+KFnqp03/pYcS7K82g4Rpeh8FQ"
Vary: Accept-Encoding
Date: Wed, 09 Sep 2026 18:23:27 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{
"status":"success",
"data":{
"ProductId":1,
"BasketId":6,
"id":9,
"quantity":3,
"createdAt":
"2026-09-09T17:56:55.264Z",
"updatedAt":
"2026-09-09T18:23:12.433Z"
}
}
normal request
250222001001
Search
0 highlights



Observation

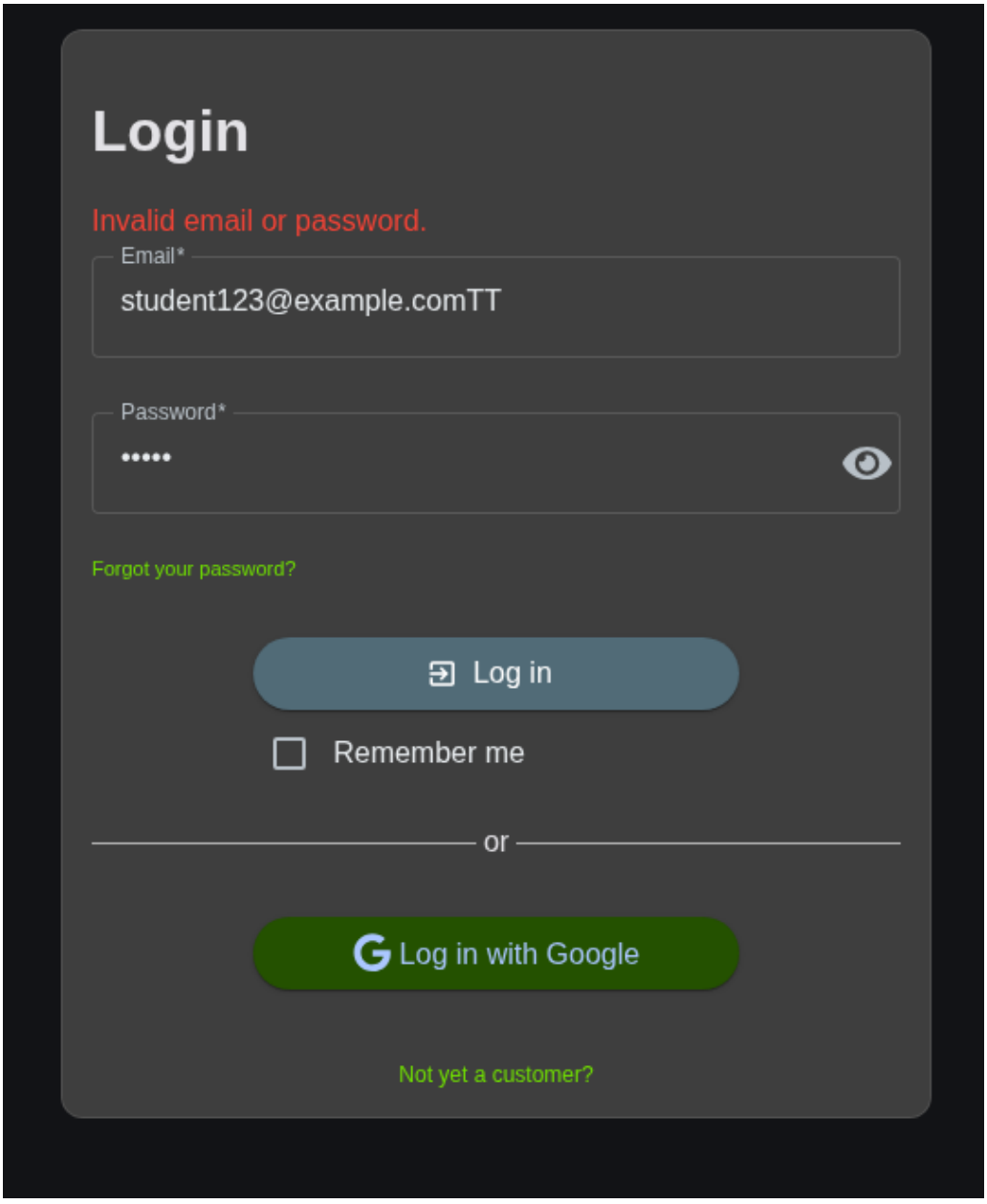
It was observed that modifying application requests can reveal weaknesses in the application's security design when proper validation, authorization, or business-logic controls are not implemented. Burp Suite helped inspect and modify requests to understand how the application responds to unexpected or unauthorized actions.

7.AUTHENTICATION FAILURE

Procedure

1. Start **OWASP Juice Shop** and open it in the browser.

2. Open **Burp Suite** and configure the browser proxy.
3. Turn **Intercept ON**.
4. Open the Juice Shop **Login** page and enter test credentials.
5. Submit the login request and capture it in Burp Suite.
6. Open **Proxy → HTTP history** and examine the authentication request.
7. Send the request to **Repeater** and inspect the parameters, response, and authentication token.
8. Test invalid credentials and observe how the application handles failed login attempts.
9. Check whether authentication and session controls are properly enforced.



Observation

It was observed that Burp Suite can capture and inspect authentication requests and responses. Testing invalid credentials helped demonstrate how weak authentication controls, improper session management, or inadequate login protections can lead to authentication failures.

8.SOFTWARE OR DATA INTEGRITY FAILURES

Procedure

1. Start OWASP Juice Shop and open it in the browser.
2. Open Burp Suite and configure the browser proxy.
3. Turn **Intercept ON**.
4. Perform an application function such as login or submitting a request.
5. Capture the request in **Proxy → HTTP history**.
6. Send the request to **Repeater**.
7. Inspect parameters, cookies, tokens, and responses.
8. Modify a request parameter and observe whether the application properly validates the changed data.

Observation

It was observed that application requests and data can be intercepted and modified using Burp Suite. Proper integrity checks and server-side validation are required to prevent unauthorized modification or manipulation of application data.

9.SECURITY LOGGING AND ALERTING FAILURES

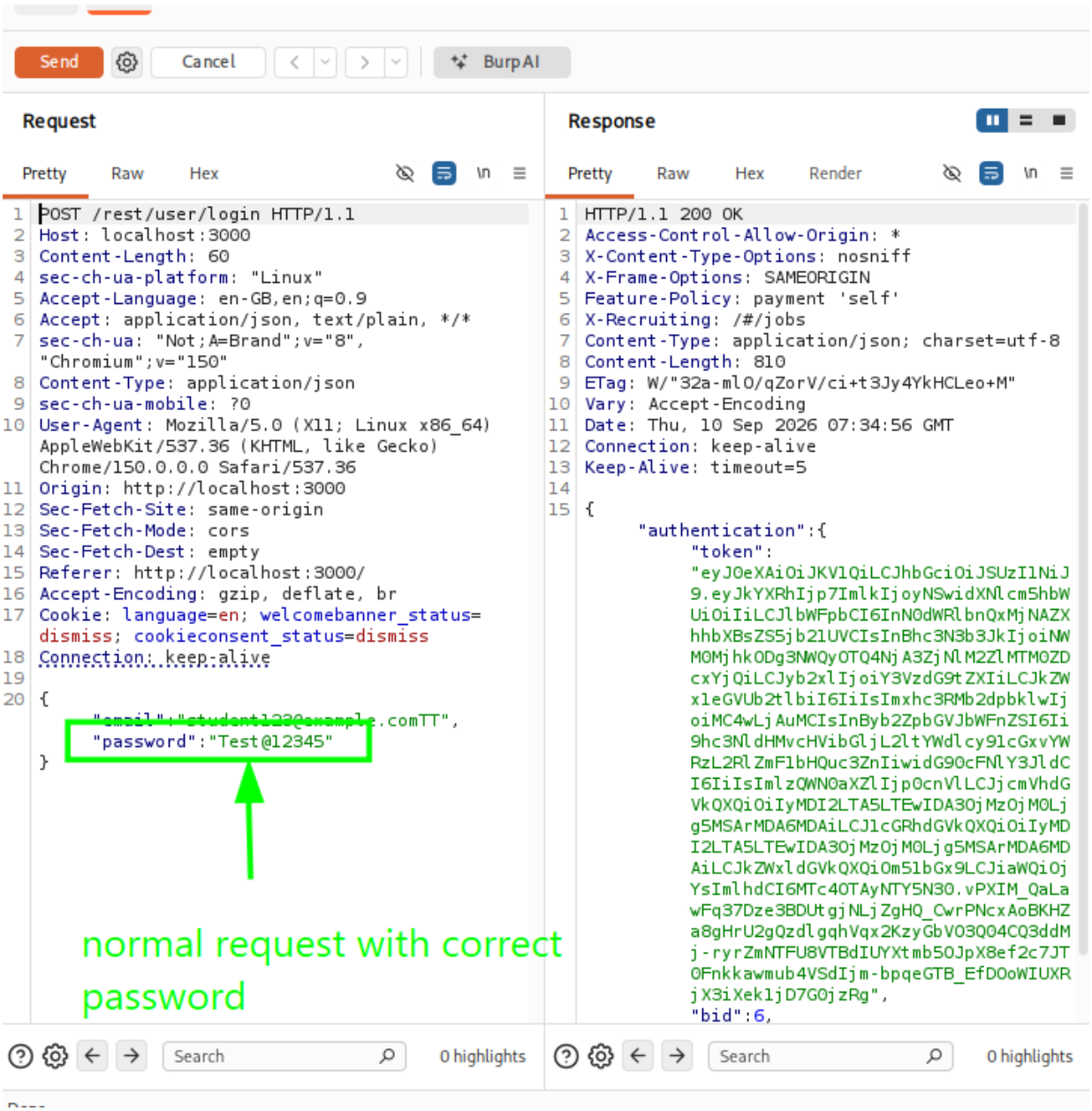
Procedure

1. Start OWASP Juice Shop and open it in the browser.
2. Configure the browser with Burp Suite.
3. Turn **Intercept ON**.
4. Perform several unsuccessful login attempts using test credentials.
5. Capture and inspect the requests and responses in Burp Suite.

6. Check whether suspicious or repeated activities generate appropriate responses or security indications.
7. Review the application's available logs, if accessible, to determine whether security events are recorded.

Observation

It was observed that repeated failed authentication requests can be identified through Burp Suite. Effective security logging and alerting are important for detecting suspicious activities such as repeated login failures and unauthorized requests.


Send
Cancel
BurpAI
Request
Pretty
Raw
Hex
Response
Pretty
Raw
Hex
Render
POST /rest/user/login HTTP/1.1
Host: localhost:3000
Content-Length: 60
sec-ch-ua-platform: "Linux"
Accept-Language: en-GB,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Origin: http://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
Connection: keep-alive
{
"email":"student123@example.comTT",
"password":"Test@12345"
}
normal request with correct password
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 810
ETag: W/"32a-mlO/qZorV/ci+t3Jy4YkHCLeo+M"
Vary: Accept-Encoding
Date: Thu, 10 Sep 2026 07:34:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{
"authentication":{
"token":
"eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InN0dWRlbnQxMjNAZXhhbXBsZS5jb21lUVCIsInBhc3N3b3JkIjoiNWM0MjhkODg3NWQyOTQ4NjA3ZjNlM2ZlMTM0ZDcxYjQiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWxleGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiOiIyMDI2LTA5LTEwIDA3OjMzOjM0Lj g5MSArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTA5LTEwIDA3OjMzOjM0Lj g5MSArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJiaWQiOjYsImlhdCI6MTc4OTAyNTY5N30.vPXIM_QaLawFq37Dze3BDUtgjNLjZgHQ_CwrPNcxAoBKHZa8gHrU2gQzdlgqhVqx2KzyGbVO3Q04CQ3ddMj-ryrZmNTFU8VTBdIUYXtmb5OJpX8ef2c7JT0Fnkkawmub4VSdIjm-bpqeGTB_EfDOoWIUXRjX3iXek1jD7G0jzRg",
"bid":6,
Search
0 highlights
Search
0 highlights

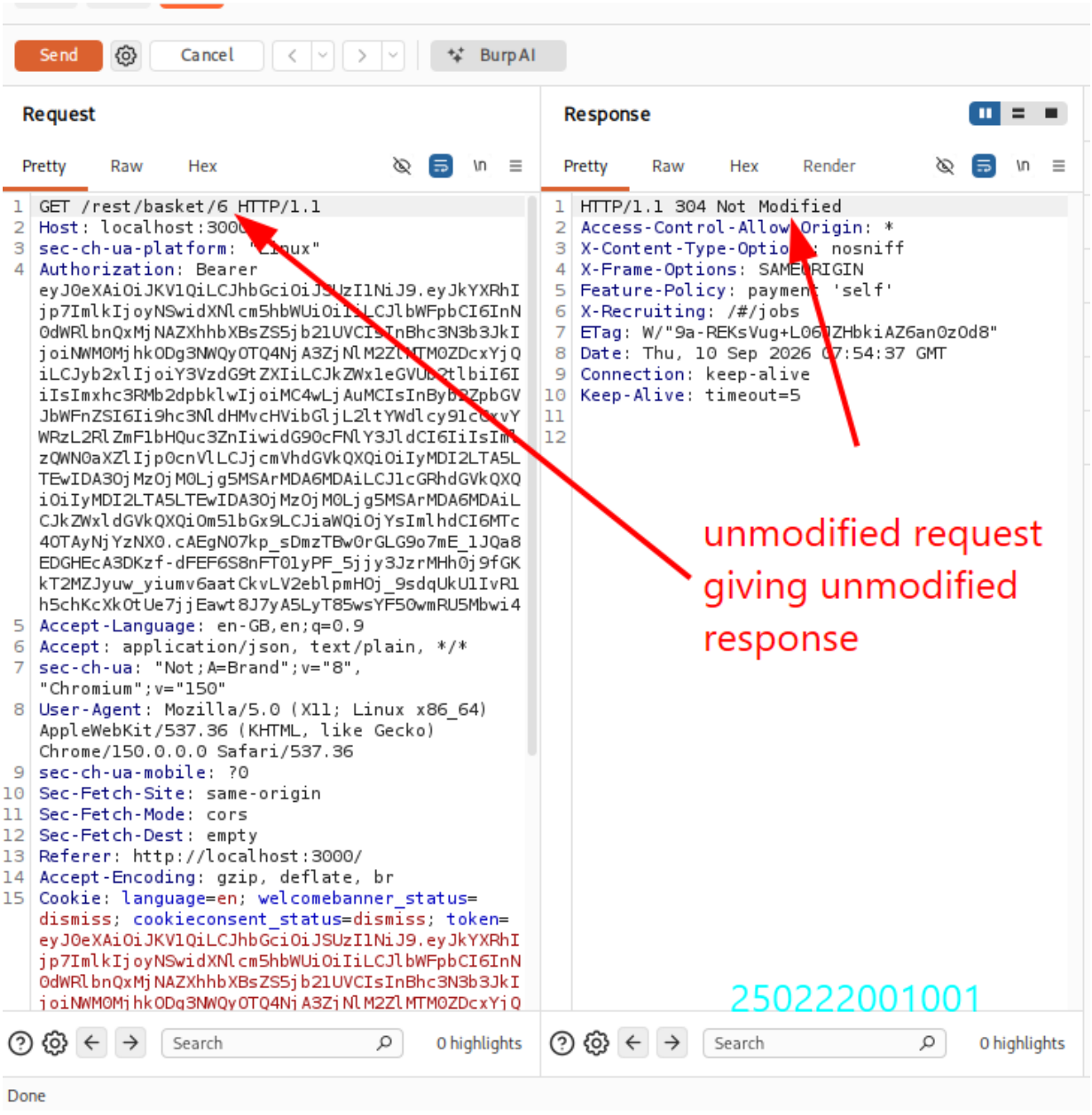


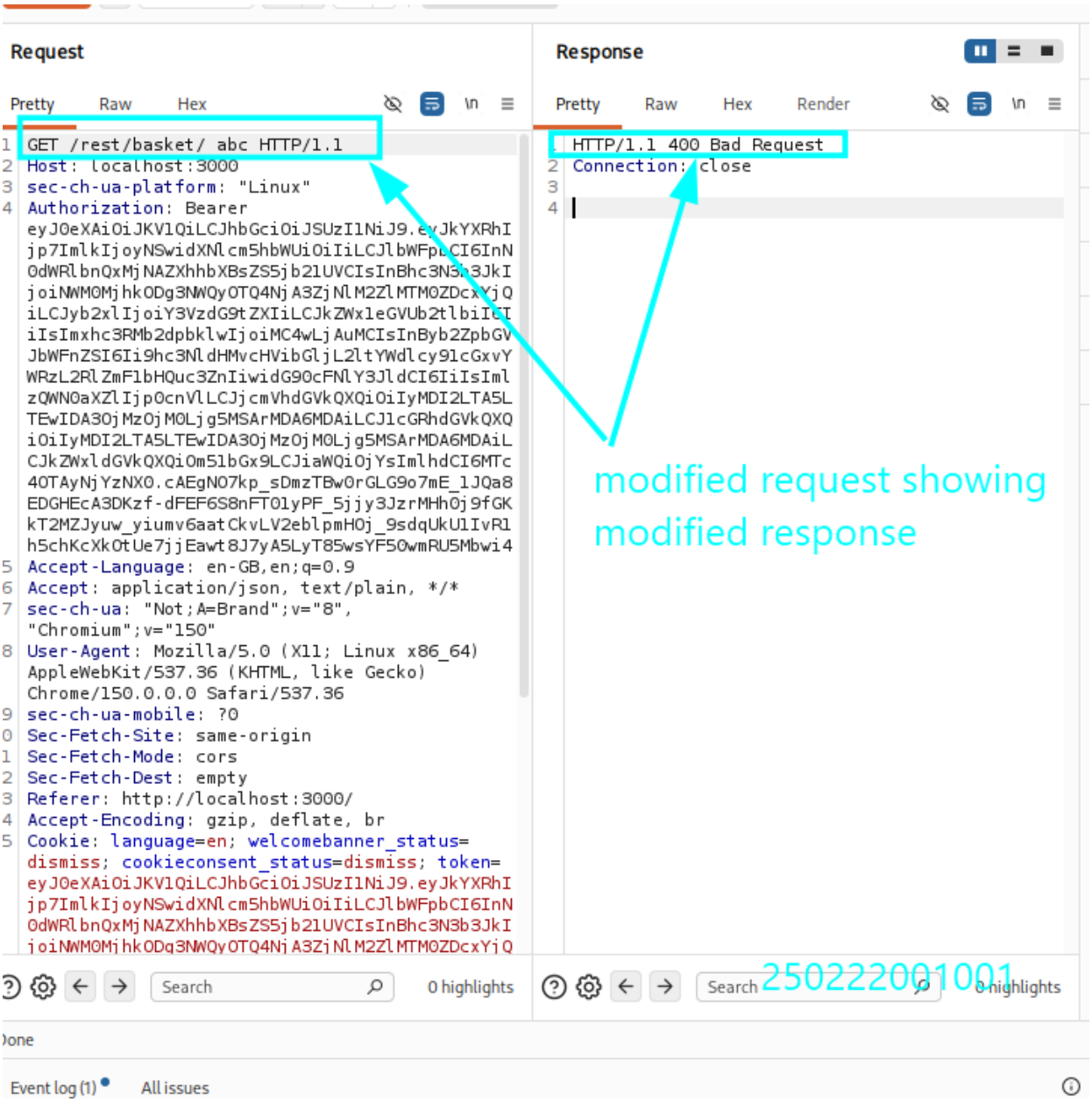
10.MISHANDLING OF EXCEPTIONAL CONDITIONS

Procedure

1. Start OWASP Juice Shop and open it in the browser.
2. Configure Burp Suite as the browser proxy.
3. Turn **Intercept ON**.
4. Perform a normal application request.
5. Capture the request and send it to **Repeater**.

6. Modify input parameters with unexpected or invalid values.
7. Send the modified request and observe the server response.
8. Check whether the application handles the error safely without exposing sensitive information.





Observation

It was observed that unexpected or invalid inputs may cause the application to return error responses. Improper handling of exceptional conditions can expose unnecessary technical information or cause unexpected application behavior.